suaRAkyat — Software Design Document (SDD) v1.0

Oct 25, 2026 · @nand

1. Pendahuluan

Dokumen ini menjelaskan **bagaimana** suaRAkyat dibangun: arsitektur, skema data, hak akses, kontrak API, dan algoritma skor. **Apa** yang dibangun dan **kenapa** ada di PRD v0.3; kebutuhan yang terukur ada di SRS.

1.1 Ruang lingkup

SDD ini mencakup semua fitur MVP dan paket anti-curang di PRD v0.3. Fitur "sebaiknya ada" dirancang cukup agar skemanya tidak perlu dirombak nanti, dan ditandai *(sebaiknya ada)*.

1.2 Pembaca dan cara memakai

Pembaca	Bagian utama
A · Frontend & Desain	3, 6, 7, 9
B · Backend & Integrasi	3, 4, 5, 6, 11, 12
C · Data & Skor	4, 6, 7, 8, 10, 13

Bagian 6 (kontrak API) dan 7 (tipe data) adalah **kesepakatan antar anggota**. Perubahan di dua bagian ini wajib diumumkan ke tim dan disetujui pemilik kedua sisi sebelum di-merge.

1.3 Referensi

- PRD suaRAkyat v0.3, terutama bagian 4 (hak akses), 7 (aturan bisnis), 8 (rapor), 9 (skor prioritas), 10 (data).
- SRS suaRAkyat (menyusul).
- Ketentuan karya WIA MIA 2026 dari Technical Meeting 17 Oktober 2026.

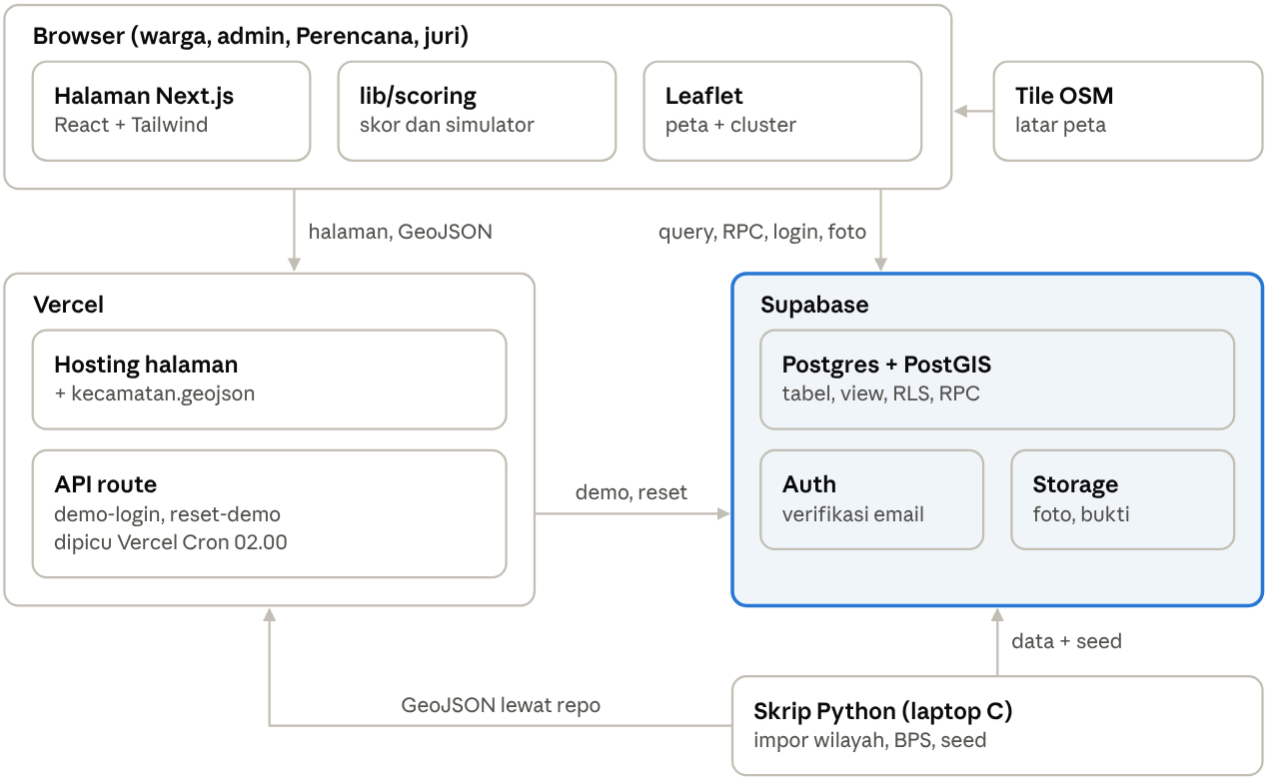
1.4 Istilah teknis

Istilah	Arti
RPC	Fungsi Postgres yang dipanggil dari frontend lewat `supabase.rpc()`
RLS	Row Level Security: aturan akses per baris yang ditegakkan database
View	Tabel virtual hasil query; dipakai untuk mengekspos data publik tanpa kolom sensitif
Isu wilayah	Satu kecamatan × satu kategori; unit skor prioritas
Waktu acuan	Titik waktu untuk menghitung umur laporan dan keterlambatan; default `now()`

2. Arsitektur sistem

suaRAkyat adalah aplikasi Next.js di Vercel yang berbicara langsung dengan Supabase; tidak ada server backend terpisah. Logika yang menyangkut keamanan dan integritas data ada di database (RPC + RLS), sedangkan logika tampilan dan simulasi berjalan di browser.

Browser langsung ke Supabase; Vercel hanya menyajikan halaman dan 2 API route



arsitektur suaRAkyat · browser, Vercel, Supabase, skrip data

Komponen	Teknologi	Tanggung jawab	Pemilik
Frontend web	Next.js (App Router) + TypeScript + Tailwind, di Vercel	Semua halaman, peta, perhitungan skor dan simulator di browser	A (C untuk `lib/scoring`)
API route	Next.js route handler	Login akun demo, endpoint cron reset demo	B
Database	Supabase Postgres + PostGIS	Tabel, view, RLS, fungsi RPC	B (skema, RLS, RPC laporan), C (RPC rapor dan prioritas)
Auth	Supabase Auth	Daftar, login, verifikasi email, sesi	B
Storage	Supabase Storage	Foto laporan dan foto bukti	B
Penjadwal	Vercel Cron	Reset data demo tiap malam	B

Komponen	Teknologi	Tanggung jawab	Pemilik
Skrip data	Python (pandas, geopandas), dijalankan di laptop	Impor wilayah dan BPS, generator seed	C
Peta dasar	Leaflet + tile OpenStreetMap	Latar peta; atribusi "© OpenStreetMap contributors" wajib tampil	A

2.1 Alur utama

1. **Warga melapor.** Browser mengompres foto dan membuang EXIF → unggah ke Storage → `similar_reports` → `create_report` → warga menerima kode `SR-XXXXXX`.
2. **Admin mengubah status.** `update_report_status` memvalidasi aturan, menulis baris timeline baru, dan view publik langsung menampilkan status terbaru.
3. **Dashboard prioritas.** Browser memanggil `priority_factors` dan membaca `priority_weights` sekali. Setelah itu skor, peringkat, simulator, dan "Kenapa belum?" dihitung `lib/scoring` di browser tanpa panggilan server lagi.
4. **Rapor Instansi.** `agency_report_card` dihitung di database setiap kali halaman dibuka.
5. **Reset demo.** Vercel Cron memanggil `/api/cron/reset-demo` setiap malam, yang memanggil `reset_demo_data()` dengan service role.

3. Struktur proyek dan konvensi

Satu repo untuk semua kode dan data, dengan pemilik yang jelas per folder supaya tiga orang bisa bekerja paralel tanpa saling menimpa.

```
suarakyat/
├─ app/                          # halaman Next.js (A)
|  ├─ page.tsx                   # Beranda
|  ├─ peta/  lapor/  laporan/[kode]/
|  ├─ rapor/  rapor/cara-menilai/
|  ├─ prioritas/  sumber-data/  masuk/
|  ├─ admin/laporan/  admin/laporan/[kode]/
|  └─ api/                       # (B)
|     ├─ demo-login/route.ts
|     └─ cron/reset-demo/route.ts
├─ components/                   # komponen UI (A)
├─ lib/
|  ├─ supabase/                  # client browser dan server (B)
|  ├─ scoring/                   # normalisasi, skor, penjelasan, simulator
```

```
(C)
|  ├─ errors.ts                 # peta kode error ke pesan (B)
|  └─ types.ts                  # tipe bersama = kontrak data (semua)
├─ public/data/kecamatan.geojson  # batas wilayah statis (C)
├─ supabase/
|  ├─ migrations/
|  |  ├─ 0001_schema.sql        # (B)
|  |  ├─ 0002_views_rls.sql     # (B)
|  |  ├─ 0003_fn_laporan.sql    # (B)
|  |  ├─ 0004_fn_rapor.sql      # (C)
|  |  └─ 0005_fn_prioritas.sql  # (C)
|  └─ seed/                     # hasil generator seed (C)
├─ scripts/                     # impor data dan generator seed, Python (C)
├─ data/raw/  data/processed/   # salinan lokal dataset (C)
└─ tests/                       # Vitest untuk lib/scoring (C)
```

3.1 Aturan kerja

Hal	Aturan
Branch	`main` = produksi. Fitur di `feat/<peran>-<fitur>`, misalnya `feat/c-rapor`. Perbaikan di `fix/<topik>`.
Pull request	Setiap PR dicek minimal satu anggota lain. Vercel otomatis membuat preview per PR.
Commit	Format `feat:`, `fix:`, `docs:`, `data:` + kalimat singkat bahasa Indonesia.
Migrasi	File SQL baru selalu ditambahkan dengan nomor berikutnya; file yang sudah di-merge tidak diedit.
Penamaan	Tabel dan kolom `snake_case` bahasa Indonesia sesuai PRD; komponen React `PascalCase`; fungsi TypeScript `camelCase`.
Kontrak	Perubahan `lib/types.ts` atau bentuk output RPC wajib diumumkan di grup dan disetujui pemilik kedua sisi.
Selesai	Fitur dianggap selesai jika sudah di `main`, ter-deploy, dan dicoba di ponsel.

3.2 Variabel lingkungan

Nama	Dipakai di	Rahasia?
`NEXT_PUBLIC_SUPABASE_URL`	Browser dan server	Tidak
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Browser dan server	Tidak (dilindungi RLS)
`SUPABASE_SERVICE_ROLE_KEY`	Hanya API route dan skrip	**Ya, jangan pernah ke browser atau repo**
`CRON_SECRET`	`/api/cron/reset-demo`	Ya
`DEMO_WARGA_EMAIL`, `DEMO_WARGA_PASSWORD`	`/api/demo-login`	Ya
`DEMO_ADMIN_EMAIL`, `DEMO_ADMIN_PASSWORD`	`/api/demo-login`	Ya

File `.env.local` masuk `.gitignore`; contoh nama variabel tanpa nilai disimpan di `.env.example`.

4. Desain data

Skema terdiri dari 14 tabel di schema `public`, empat view publik, dan schema `seed` untuk reset demo. Skema di bawah adalah isi `0001_schema.sql` dan menjadi acuan tunggal; PRD bagian 12 hanya ringkasannya.

4.1 Skema SQL

```
create extension if not exists postgis;

create type peran          as enum ('warga','admin');
create type tingkat_urgensi as enum ('tinggi','sedang','rendah');
create type status_laporan as enum
('menunggu','diverifikasi','diproses','selesai','ditolak','dibatalkan');
create type alasan_tolak   as enum
('duplikat','tidak_valid','konten_melanggar');

create table app_config (            -- angka yang bisa disetel tanpa deploy
  kunci text primary key,
  nilai jsonb not null
);
-- isi awal: radius_serupa_m=100, batas_upvote=10, ambang_tolak_tetap=0.25,
--           ambang_tolak_min=0.10, sampel_min_rapor=10,
```

```
seed_anchor='<timestamptz>'

create table agencies (
  id smallint generated always as identity primary key,
  nama text not null unique,
  is_simulated boolean not null default true
);

create table categories (
  id smallint generated always as identity primary key,
  nama text not null unique,
  instansi_id smallint not null references agencies(id),
  urgensi tingkat_urgensi not null,
  sla_respons_jam int not null check (sla_respons_jam > 0),
  sla_selesai_jam int not null check (sla_selesai_jam > 0)
);

create table regions (
  id int generated always as identity primary key,
  kode_wilayah text not null unique,
  nama text not null,
  kota text not null,
  geom geometry(MultiPolygon, 4326) not null,
  penduduk int not null check (penduduk > 0),
  luas_km2 numeric(8,2) not null check (luas_km2 > 0),
  persen_miskin numeric(5,2) not null,
  sumber text not null
);
create index regions_geom_idx on regions using gist (geom);

create table profiles (               -- 1:1 dengan auth.users, dibuat oleh
trigger saat daftar
  id uuid primary key references auth.users(id) on delete cascade,
  nama text not null,
  role peran not null default 'warga',
  instansi_id smallint references agencies(id),
  kelola_prioritas boolean not null default false,
  is_demo boolean not null default false,
  check (role = 'admin' or (instansi_id is null and kelola_prioritas =
false))
);

create table reports (
  id uuid primary key default gen_random_uuid(),
  kode text not null unique,                        -- 'SR-' + 6 karakter
  user_id uuid not null references profiles(id),
```

```
  category_id smallint not null references categories(id),
  region_id int not null references regions(id),
  deskripsi text not null check (char_length(deskripsi) between 20 and 500),
  foto text[] not null check (cardinality(foto) between 1 and 3),
  lokasi geography(Point, 4326) not null,
  status status_laporan not null default 'menunggu',
  perkiraan_selesai date,                          -- (sebaiknya ada)
  kategori_diubah boolean not null default false,  -- (sebaiknya ada)
  created_at timestamptz not null default now(),
  responded_at timestamptz,
  resolved_at timestamptz,
  is_simulated boolean not null default false
);
create index reports_lokasi_idx on reports using gist (lokasi);
create index reports_filter_idx on reports (status, category_id, region_id);

create table report_updates (         -- sumber timeline; satu baris per
perubahan status
  id bigint generated always as identity primary key,
  report_id uuid not null references reports(id) on delete cascade,
  admin_id uuid references profiles(id),           -- null = aksi pelapor
(batal)
  status_baru status_laporan not null,
  alasan_tolak alasan_tolak,
  duplikat_dari uuid references reports(id),
  catatan text,
  bukti_url text,
  created_at timestamptz not null default now(),
  check (status_baru <> 'ditolak' or alasan_tolak is not null),
  check (alasan_tolak <> 'duplikat' or duplikat_dari is not null),
  check (alasan_tolak not in ('tidak_valid','konten_melanggar') or catatan is
not null),
  check (status_baru <> 'selesai' or bukti_url is not null)
);
create index report_updates_report_idx on report_updates (report_id,
created_at);

create table upvotes (
  report_id uuid references reports(id) on delete cascade,
  user_id uuid references profiles(id) on delete cascade,
  created_at timestamptz not null default now(),
  primary key (report_id, user_id)
);

create table ratings (
  report_id uuid primary key references reports(id) on delete cascade,  --
```

```
satu rating per laporan
  user_id uuid not null references profiles(id),
  nilai smallint not null check (nilai between 1 and 5),
  komentar text check (char_length(komentar) <= 300),
  created_at timestamptz not null default now()
);

create table priority_weights (
  faktor text primary key
    check (faktor in
('urgensi','volume','dukungan','umur','kerentanan','kepadatan')),
  bobot numeric(4,3) not null check (bobot between 0 and 1),
  updated_by uuid references profiles(id),
  updated_at timestamptz not null default now()
);

create table weight_changes (         -- satu baris per penyimpanan, berisi
snapshot semua bobot
  id bigint generated always as identity primary key,
  bobot_lama jsonb not null,
  bobot_baru jsonb not null,
  alasan text not null check (char_length(alasan) >= 10),
  admin_id uuid not null references profiles(id),
  created_at timestamptz not null default now()
);

create table priority_overrides (     -- (sebaiknya ada)
  id bigint generated always as identity primary key,
  region_id int not null references regions(id),
  category_id smallint not null references categories(id),
  admin_id uuid not null references profiles(id),
  arah text not null check (arah in ('atas','bawah')),
  alasan text not null,
  aktif boolean not null default true,
  created_at timestamptz not null default now()
);

create table keberatan (              -- (sebaiknya ada)
  id bigint generated always as identity primary key,
  report_id uuid not null references reports(id),
  user_id uuid not null references profiles(id),
  jenis text not null check (jenis in ('banding','belum_beres')),
  alasan text not null,
  status text not null default 'menunggu' check (status in
('menunggu','diterima','ditolak')),
  diputuskan_oleh uuid references profiles(id),
```

```
  created_at timestamptz not null default now()
);

create table crm_agregat (            -- hanya jika dataset CRM DKI layak
  region_id int references regions(id),
  category_id smallint references categories(id),
  bulan date,
  jumlah_laporan int not null,
  sumber text not null,
  primary key (region_id, category_id, bulan)
);
```

Catatan: `weight_changes` menyimpan snapshot semua bobot per penyimpanan, bukan satu baris per faktor seperti ringkasan di PRD. Bentuk ini lebih mudah ditampilkan sebagai riwayat keputusan.

4.2 Relasi

Tabel	Merujuk ke	Kardinalitas
categories	agencies	Banyak kategori ke satu instansi (MVP: 1 kategori per instansi)
profiles	auth.users, agencies	1:1 dengan akun; admin ke satu instansi
reports	profiles, categories, regions	Banyak laporan ke satu pelapor, kategori, kecamatan
report_updates	reports, profiles	Banyak baris timeline per laporan
upvotes	reports, profiles	Unik per pasangan laporan–akun
ratings	reports, profiles	Maksimal satu per laporan
weight_changes, priority_weights	profiles	Dicatat per Perencana
priority_overrides, crm_agregat	regions, categories	Per isu wilayah

4.3 View publik

Publik tidak pernah membaca tabel `reports` langsung, karena tabel itu berisi `user_id` dan isi laporan yang mungkin disembunyikan. Semua bacaan publik lewat view berikut, yang dimiliki `postgres` sehingga bisa membaca tabel dasar tapi hanya mengekspos kolom

aman.

View	Isi	Aturan penting
`v_laporan_publik`	kode, kategori, instansi, kecamatan, kota, status, lat, lng, deskripsi, foto, foto_bukti, alasan_tolak, inisial_pelapor, jumlah_upvote, created_at, responded_at, resolved_at, perkiraan_selesai, batas_respons, batas_selesai, is_simulated	Deskripsi dan foto dikosongkan jika alasan tolak `konten_melanggar`; status `dibatalkan` tidak muncul; foto_bukti hanya jika status `selesai`
`v_timeline_publik`	kode, status_baru, alasan_tolak, catatan, bukti_url, pelaku ("Pelapor" atau "Admin · nama instansi"), created_at	Tidak mengekspos `admin_id`
`v_riwayat_bobot`	created_at, bobot_lama, bobot_baru, alasan, pelaku ("Perencana · Bappeda Kota (simulasi)")	Tidak mengekspos `admin_id`
`v_laporan_admin`	Semua kolom `reports` + kategori, kecamatan, batas_respons, batas_selesai, terlambat	`security_invoker = true`, jadi RLS tabel `reports` tetap berlaku

4.4 Schema seed

Schema `seed` berisi salinan `reports`, `report_updates`, `upvotes`, dan `ratings` hasil generator (bagian 10), plus `seed_anchor` di `app_config`. Saat reset, data disalin ulang ke `public` dengan semua waktu digeser sebesar `now() − seed_anchor`. Akibatnya umur laporan dan status terlambat selalu sama persis setiap selesai reset, sehingga angka di video dan proposal konsisten.

5. Keamanan dan hak akses

Aturan akses ditegakkan di database, bukan di tampilan. Klien tidak pernah menulis langsung ke tabel penting; semua penulisan lewat RPC yang memeriksa peran, kepemilikan, dan aturan bisnis.

5.1 Autentikasi

- Email + password lewat Supabase Auth, dengan opsi **Confirm email** aktif. Akun yang belum verifikasi tidak bisa login, jadi tidak bisa melapor, upvote, atau memberi rating.
- Trigger `handle_new_user` pada `auth.users` membuat baris `profiles` dengan peran `warga` dan nama dari metadata pendaftaran.
- Akun admin instansi, Perencana, dan demo dibuat skrip memakai service role (`auth.admin.createUser` dengan `email_confirm: true`), lalu profilnya diisi peran dan instansi.
- Fungsi bantu SQL untuk kebijakan: `is_admin()`, `my_instansi()`, `is_perencana()`. Ketiganya `security definer stable` dan membaca `profiles` milik `auth.uid()`.

5.2 Matriks RLS

Tabel	Baca (SELECT)	Tulis
`app_config`, `agencies`, `categories`, `regions`, `priority_weights`, `crm_agregat`	Publik	Tidak dari klien; hanya migrasi, skrip, atau RPC
`profiles`	Baris sendiri	Update kolom `nama` milik sendiri
`reports`	Pelapor (miliknya), admin (kategori instansinya), Perencana (semua). Publik lewat `v_laporan_publik`	Hanya lewat RPC
`report_updates`	Admin instansi dan Perencana. Publik lewat `v_timeline_publik`	Hanya lewat RPC
`upvotes`	Baris milik sendiri. Jumlah publik lewat view	Hanya lewat `toggle_upvote`
`ratings`	Pelapor (miliknya)	Hanya lewat `rate_report`
`weight_changes`	Publik lewat `v_riwayat_bobot`	Hanya lewat `save_weights`
`priority_overrides`	Publik, baris `aktif` saja	Hanya lewat RPC *(sebaiknya ada)*

Tabel	Baca (SELECT)	Tulis
`keberatan`	Pelapor (miliknya), Perencana	Hanya lewat RPC *(sebaiknya ada)*

5.3 Aturan RPC

- Semua RPC ditulis `security definer` dengan `set search_path = public`, lalu memeriksa `auth.uid()` dan peran di baris pertama.
- `execute` dicabut dari `public`. RPC tulis hanya diberikan ke `authenticated`; RPC baca (`agency_report_card`, `priority_factors`, `stats_beranda`) diberikan ke `anon` dan `authenticated`.
- `reset_demo_data()` hanya bisa dipanggil `service_role`.

5.4 Storage

Bucket	Baca	Unggah	Batas
`foto-laporan`	Publik	Pengguna login, hanya ke folder `<uid>/` miliknya	JPG/PNG/WebP, maks 5 MB
`bukti`	Publik	Admin, hanya ke folder `<uid>/` miliknya	JPG/PNG/WebP, maks 5 MB

Foto dikompres dan EXIF-nya dibuang di browser sebelum diunggah (gambar digambar ulang lewat canvas). File diberi nama UUID acak, bukan nama asli dari perangkat.

5.5 Akun demo

- **Warga demo:** hak sama dengan warga biasa.
- **Admin demo:** terikat ke Dinas Bina Jalan (simulasi), dan hanya bisa mengubah laporan simulasi atau laporan buatan akun demo. Aturan ini dicek di `update_report_status` (error `DEMO_TERBATAS`).
- Semua perubahan oleh akun demo dihapus saat reset malam.

5.6 Di mana setiap aturan anti-curang ditegakkan

Aturan (PRD 7.5)	Ditegakkan di
Verifikasi email	Pengaturan Supabase Auth

Aturan (PRD 7.5)	Ditegakkan di
Satu upvote per akun, tidak untuk laporan sendiri	Primary key `upvotes` + cek di `toggle_upvote`
Maksimal 10 upvote dihitung	`priority_factors`
Tiga alasan tolak, catatan wajib, cek duplikat otomatis	Constraint `report_updates` + `update_report_status`
Isi "Konten melanggar" disembunyikan	`v_laporan_publik`
Laporan Ditolak tetap publik	`v_laporan_publik` tidak memfilter status `ditolak`
Peringatan penolakan dan huruf maksimal C	`agency_report_card`
Bobot hanya diubah Perencana, alasan wajib, riwayat publik	`save_weights` + `v_riwayat_bobot`
Batalkan hanya oleh pelapor saat Menunggu	`cancel_report`

6. Kontrak API

Ada 22 operasi MVP dalam lima jenis: query langsung ke view/tabel, RPC, file statis, Storage, dan API route. Tidak ada REST API buatan sendiri; frontend memanggil Supabase lewat `supabase-js`.

6.1 Ringkasan operasi

ID	Operasi	Jenis	Dipanggil dari	Akses	Pemilik
Q1	Daftar laporan untuk peta	Query `v_laporan_publik`	Beranda, Peta	Publik	B
Q2	Detail laporan + timeline	Query `v_laporan_publik`, `v_timeline_publik`	Detail & Lacak	Publik	B
Q3	Daftar laporan instansi	Query `v_laporan_admin`	Admin	Admin	B
Q4	Kategori dan instansi	Query `categories` + `agencies`	Form, filter	Publik	B
Q5	Bobot yang berlaku	Query `priority_weights`	Prioritas	Publik	C
Q6	Riwayat bobot	Query `v_riwayat_bobot`	Prioritas	Publik	C

ID	Operasi	Jenis	Dipanggil dari	Akses	Pemilik
F1	Batas kecamatan	File statis `/data/kecamatan.geojson`	Peta, Prioritas	Publik	C
S1	Unggah foto laporan/bukti	Storage	Form lapor, Admin	Login	B
R1	`stats_beranda`	RPC	Beranda	Publik	B
R2	`similar_reports`	RPC	Form lapor	Warga	B
R3	`create_report`	RPC	Form lapor	Warga	B
R4	`toggle_upvote`	RPC	Detail, laporan serupa	Warga	B
R5	`cancel_report`	RPC	Detail	Pelapor	B
R6	`update_report_status`	RPC	Admin	Admin instansi	B
R7	`rate_report`	RPC	Detail	Pelapor	B
R8	`agency_report_card`	RPC	Rapor	Publik	C
R9	`priority_factors`	RPC	Prioritas, "Kenapa belum?"	Publik	C
R10	`save_weights`	RPC	Prioritas (mode edit)	Perencana	B
R11	`reset_demo_data`	RPC	Dipanggil A2	service_role	B
A1	`POST /api/demo-login`	API route	Beranda	Publik	B
A2	`GET /api/cron/reset-demo`	API route + Vercel Cron	Sistem	`CRON_SECRET`	B
L1	Skor, simulator, "Kenapa belum?"	Fungsi `lib/scoring` di browser	Prioritas, Detail	—	C

(Sebaiknya ada): `change_category`, `submit_keberatan`, `decide_keberatan`, `set_override`. Bentuknya mengikuti pola R5–R7 dan ditulis saat fiturnya dikerjakan.

6.2 Konvensi error

RPC menolak permintaan dengan `raise exception '<KODE>' using errcode = 'P0001';`. Di frontend, `error.message` berisi kode itu dan diterjemahkan oleh `lib/errors.ts`. Detail tambahan dikirim lewat `using detail = '...'`.

Kode	Pesan untuk pengguna	Dipakai di
`TIDAK_LOGIN`	Silakan masuk dulu.	Semua RPC tulis
`INPUT_TIDAK_VALID`	Periksa kembali isian Anda.	Semua RPC tulis
`LAPORAN_TIDAK_ADA`	Kode laporan tidak ditemukan.	R4–R7
`LUAR_WILAYAH`	Lokasi di luar wilayah layanan (DKI Jakarta daratan).	R3
`BUKAN_PEMILIK`	Aksi ini hanya untuk pelapor laporan ini.	R5, R7
`BUKAN_INSTANSI`	Laporan ini bukan milik instansi Anda.	R6
`BUKAN_PERENCANA`	Hanya Perencana yang bisa mengubah bobot.	R10
`TRANSISI_TIDAK_VALID`	Perubahan status ini tidak diizinkan.	R5, R6
`ALASAN_WAJIB`	Pilih alasan dan isi catatan penolakan.	R6
`BUKTI_WAJIB`	Lampirkan foto bukti penyelesaian.	R6
`DUPLIKAT_TIDAK_VALID`	Laporan rujukan harus berkategori sama, masih aktif, dan berjarak paling jauh 100 m.	R6
`LAPORAN_TIDAK_AKTIF`	Laporan ini sudah tidak aktif.	R4
`UPVOTE_SENDIRI`	Anda tidak bisa mendukung laporan sendiri.	R4
`BELUM_SELESAI`	Rating hanya bisa diberikan setelah laporan selesai.	R7
`SUDAH_DIRATING`	Laporan ini sudah diberi rating.	R7
`BOBOT_TIDAK_VALID`	Bobot harus 0–1 dan tidak boleh semuanya nol.	R10
`ALASAN_TERLALU_PENDEK`	Alasan minimal 10 karakter.	R10
`DEMO_TERBATAS`	Akun demo hanya bisa mengubah laporan simulasi.	R6

6.3 Detail RPC laporan (B)

Semua contoh JSON di bawah adalah ilustrasi bentuk data, bukan data nyata.

R3 · `create_report`

```
create_report(p_category_id smallint, p_deskripsi text, p_foto text[],
              p_lat float8, p_lng float8) returns jsonb
```

1. Cek login; validasi panjang deskripsi (20–500) dan jumlah foto (1–3), jika gagal `INPUT_TIDAK_VALID`.
2. Cari kecamatan dengan `ST_Contains(geom, ST_SetSRID(ST_MakePoint(p_lng, p_lat), 4326))`; jika tidak ada, `LUAR_WILAYAH`.
3. Buat kode `SR-` + 6 karakter dari alfabet 32 huruf/angka tanpa O, 0, I, 1; ulangi jika bentrok.
4. Simpan laporan, lalu tulis baris timeline pertama berstatus `menunggu` dengan `admin_id = null`.

```
{ "kode": "SR-7KQ2MX", "status": "menunggu", "kecamatan": "Contoh A", "kota":
"Jakarta Selatan",
  "instansi": "Dinas Bina Jalan (simulasi)", "batas_respons": "2026-11-
03T10:00:00+07:00" }
```

R2 · `similar_reports`

```
similar_reports(p_category_id smallint, p_lat float8, p_lng float8) returns
jsonb
```

Laporan berkategori sama, status `menunggu` / `diverifikasi` / `diproses`, dalam radius `app_config.radius_serupa_m` memakai `ST_DWithin` pada geography; maksimal 5, urut dari yang terdekat.

```
[ { "kode": "SR-3HD9TA", "jarak_m": 42, "status": "diproses",
"jumlah_upvote": 7,
    "foto": "https://<project>.supabase.co/storage/v1/object/public/foto-
laporan/...", "created_at": "2026-10-28T08:12:00+07:00" } ]
```

R4 · `toggle_upvote(p_kode text) returns jsonb` — hanya untuk laporan aktif dan bukan milik sendiri; memanggil dua kali membatalkan dukungan.

```
{ "upvoted": true, "jumlah_upvote": 8 }
```

R5 · `cancel_report(p_kode text) returns jsonb` — hanya pelapor, hanya dari status `menunggu`. Menulis timeline `dibatalkan`.

R6 · `update_report_status`

```
update_report_status(p_kode text, p_status status_laporan,
                     p_catatan text default null, p_bukti_url text default
null,
                     p_alasan alasan_tolak default null, p_duplikat_kode text
default null,
                     p_perkiraan date default null) returns jsonb
```

Dari	Boleh ke	Syarat
`menunggu`	`diverifikasi`, `ditolak`	Ditolak wajib alasan; catatan wajib untuk `tidak_valid` dan `konten_melanggar`
`diverifikasi`	`diproses`, `ditolak`	Sama seperti di atas
`diproses`	`selesai`	`p_bukti_url` wajib
`selesai`, `ditolak`, `dibatalkan`	—	Final

- Admin harus satu instansi dengan kategori laporan (`BUKAN_INSTANSI`); admin demo terkena aturan `DEMO_TERBATAS`.
- `responded_at` diisi saat pertama kali keluar dari `menunggu` (termasuk ditolak); `resolved_at` diisi saat `selesai`.
- Untuk `duplikat`: laporan rujukan harus berbeda, berkategori sama, masih aktif, dan dalam radius serupa. Upvote dipindah ke rujukan (`insert ... on conflict do nothing`, lalu hapus yang lama).

```
{ "kode": "SR-7KQ2MX", "status": "diverifikasi", "responded_at": "2026-11-
02T14:05:00+07:00", "resolved_at": null }
```

R7 · `rate_report(p_kode text, p_nilai smallint, p_komentar text default null) returns jsonb` — hanya pelapor, hanya jika status `selesai`, sekali per laporan.

R10 · `save_weights(p_bobot jsonb, p_alasan text) returns jsonb` — hanya Perencana. `p_bobot` wajib berisi tepat enam kunci faktor, setiap nilai ≥ 0, jumlah > 0. Bobot dinormalisasi ke total 1, snapshot lama dan baru dicatat di `weight_changes`.

```
{ "bobot": { "urgensi": 0.30, "volume": 0.20, "dukungan": 0.05, "umur": 0.20,
"kerentanan": 0.15, "kepadatan": 0.10 },
  "riwayat_id": 12 }
```

R1 · `stats_beranda() returns jsonb`

```
{ "total_laporan": 912, "persen_selesai": 61.4, "terlambat_aktif": 87,
  "per_kategori": [ { "kategori": "Jalan rusak", "jumlah": 301 } ],
"is_simulated": true }
```

R11 · **`reset_demo_data() returns jsonb`** — hanya service role. Menghapus semua laporan, timeline, upvote, rating, dan keberatan yang dibuat atau diubah akun demo, lalu menyalin ulang schema `seed` dengan waktu digeser (bagian 4.4). Laporan dari akun warga sungguhan tidak disentuh.

6.4 Detail RPC rapor dan prioritas (C)

R8 · **`agency_report_card(p_dari date default null, p_sampai date default null) returns jsonb`**. Rumusnya ada di bagian 8; tipenya `RaporResponse` di bagian 7.

```
{ "periode": { "dari": "2026-05-01", "sampai": "2026-11-01" },
"is_simulated": true,
  "instansi": [ { "id": 1, "nama": "Dinas Kebersihan (simulasi)", "sampel":
95,
      "R": 0.84, "S": 0.63, "P": 0.72, "K": 0.75, "nilai": 73.5,
      "huruf_asli": "B", "huruf": "B", "tingkat_penolakan": 0.10,
"peringatan_penolakan": false,
      "median_respons_jam": 9.5, "terlambat_aktif": 12, "keberatan_diterima":
0,
      "per_kategori": [ { "kategori": "Sampah menumpuk", "sampel": 95, "R":
0.84, "S": 0.63, "P": 0.72 } ] } ] }
```

R9 · **`priority_factors(p_category_id smallint default null, p_sumber text default 'simulasi') returns jsonb`**. Mengembalikan nilai **mentah** enam faktor untuk setiap isu wilayah yang punya minimal satu laporan aktif. Normalisasi dan pembobotan tidak dilakukan di sini, tapi di `lib/scoring` (bagian 8). Dengan `p_sumber = 'crm'`, volume diambil dari `crm_agregat` dan umur bernilai `null`.

```
{ "sumber": "simulasi", "waktu_acuan": "2026-11-01T00:00:00+07:00",
  "isu": [ { "id": "12-1", "kode_wilayah": "31.00.01", "kecamatan": "Contoh
A", "kota": "Jakarta Selatan",
      "category_id": 1, "kategori": "Jalan rusak", "laporan_aktif": 6,
"laporan_terverifikasi": 4,
      "mentah": { "urgensi": 1.0, "volume": 1.8, "dukungan": 3.2, "umur":
1.35,
                  "kerentanan": 4.5, "kepadatan": 21500 } } ] }
```

6.5 Query langsung

```
// Q1 peta
supabase.from('v_laporan_publik')
  .select('kode,kategori,status,lat,lng,jumlah_upvote,is_simulated')
  .in('status', statusDipilih)

// Q2 detail + timeline
supabase.from('v_laporan_publik').select('*').eq('kode', kode).single()
supabase.from('v_timeline_publik').select('*').eq('kode',
kode).order('created_at')

// Q3 admin: RLS otomatis membatasi ke instansi admin
supabase.from('v_laporan_admin').select('*')
  .order('terlambat', { ascending: false }).order('batas_selesai').range(0,
49)

// Q5 dan Q6
supabase.from('priority_weights').select('faktor,bobot,updated_at')
supabase.from('v_riwayat_bobot').select('*').order('created_at', { ascending:
false })
```

6.6 Storage dan API route

S1 · Unggah foto. Path `foto-laporan/<uid>/<uuid>.webp` atau `bukti/<uid>/<uuid>.webp`. URL publik hasil unggah dikirim ke R3 atau R6.

A1 · `POST /api/demo-login`

Bagian	Isi
Body	`{ "peran": "warga" }` atau `{ "peran": "admin" }`
200	`{ "ok": true, "redirect": "/lapor" }` atau `{ "ok": true, "redirect": "/admin/laporan" }`; cookie sesi Supabase diset di server
400	`{ "error": "PERAN_TIDAK_VALID" }`

A2 · `GET /api/cron/reset-demo`

Bagian	Isi
Header	`Authorization: Bearer <CRON_SECRET>` (dikirim otomatis oleh Vercel Cron)
Jadwal	`vercel.json`: `"0 19 * * *"` (02.00 WIB). Di paket gratis Vercel, ketepatan jadwal tidak dijamin per menit.
200	`{ "reset_at": "2026-11-02T02:00:05+07:00" }`
401	Header salah atau tidak ada

7. Tipe data bersama

`lib/types.ts` adalah kontrak data dalam bentuk kode: frontend membangun tampilan dari tipe ini, backend dan C wajib mengembalikan bentuk yang sama. Pada hari pertama, C juga menaruh contoh data di `lib/mocks/rapor.json` dan `lib/mocks/prioritas.json` supaya A bisa mulai tanpa menunggu RPC selesai.

```
// lib/types.ts — ubah hanya dengan persetujuan pemilik kedua sisi

export type StatusLaporan =
  'menunggu' | 'diverifikasi' | 'diproses' | 'selesai' | 'ditolak' |
'dibatalkan';
export type AlasanTolak = 'duplikat' | 'tidak_valid' | 'konten_melanggar';
export type Faktor = 'urgensi' | 'volume' | 'dukungan' | 'umur' |
'kerentanan' | 'kepadatan';
export type Bobot = Record<Faktor, number>;
export type Huruf = 'A' | 'B' | 'C' | 'D' | 'E';

export interface LaporanPublik {          // Q1, Q2
  kode: string; kategori: string; instansi: string;
  kecamatan: string; kota: string; status: StatusLaporan;
  lat: number; lng: number;
  deskripsi: string | null;               // null jika konten_melanggar
  foto: string[];                         // kosong jika konten_melanggar
  foto_bukti: string | null;              // hanya jika selesai
  alasan_tolak: AlasanTolak | null;
  inisial_pelapor: string;
  jumlah_upvote: number;
  created_at: string; responded_at: string | null; resolved_at: string |
null;
  perkiraan_selesai: string | null;
```

```
  batas_respons: string; batas_selesai: string;
  is_simulated: boolean;
}

export interface TimelineItem {           // Q2
  kode: string; status_baru: StatusLaporan; alasan_tolak: AlasanTolak | null;
  catatan: string | null; bukti_url: string | null;
  pelaku: string;                         // "Pelapor" atau "Admin ·
<instansi>"
  created_at: string;
}

export interface RaporInstansi {          // R8
  id: number; nama: string; sampel: number;
  R: number; S: number; P: number; K: number;   // 0–1
  nilai: number;                          // 0–100, satu desimal
  huruf_asli: Huruf;                      // huruf sebelum batas maksimal C
  huruf: Huruf | null;                    // null = data belum cukup
  tingkat_penolakan: number; peringatan_penolakan: boolean;
  median_respons_jam: number | null;
  terlambat_aktif: number; keberatan_diterima: number;
  per_kategori: { kategori: string; sampel: number; R: number; S: number; P:
number }[];
}
export interface RaporResponse {
  periode: { dari: string; sampai: string };
  is_simulated: boolean;
  instansi: RaporInstansi[];
}

export interface FaktorIsu {              // R9, nilai mentah
  id: string;                             // "<region_id>-<category_id>"
  kode_wilayah: string; kecamatan: string; kota: string;
  category_id: number; kategori: string;
  laporan_aktif: number; laporan_terverifikasi: number;
  mentah: Record<Faktor, number | null>;  // umur null di mode crm
}
export interface PriorityFactorsResponse {
  sumber: 'simulasi' | 'crm';
  waktu_acuan: string;
  isu: FaktorIsu[];
}

export interface SkorIsu extends FaktorIsu {   // hasil lib/scoring di
browser
  nilai01: Record<Faktor, number>;        // setelah diubah ke skala 0–1
```

```
  kontribusi: Record<Faktor, number>;     // poin; jumlahnya sama dengan skor
  skor: number;                           // 0–100, satu desimal
  peringkat: number;                      // 1 = paling prioritas
  faktor_tidak_membedakan: Faktor[];      // faktor dengan maksimum = minimum
  penjelasan: string;                     // kalimat dari template
}

export interface KenapaBelum {
  peringkat: number; total: number;
  pendorong: { faktor: Faktor; poin: number };
  penahan: { faktor: Faktor; poin: number; maksimal: number };
  per_preset: { preset: string; peringkat: number }[];
  catatan: string | null;                 // misalnya "laporan ini belum
diverifikasi"
}
```

8. Desain modul rapor dan skor prioritas

Rapor dihitung di database karena butuh banyak baris laporan; skor prioritas dihitung di browser karena harus bereaksi seketika saat slider digeser. Rumusnya mengikuti PRD bagian 8 dan 9; bagian ini menjelaskan cara menghitungnya langkah demi langkah.

8.1 `agency_report_card` (SQL, C)

1. Ambil laporan dalam periode, kecuali status `dibatalkan` dan laporan yang ditolak sebagai `duplikat`. Laporan yang pindah kategori otomatis ikut instansi barunya.
2. Per laporan, hitung `batas_respons` dan `batas_selesai` dari SLA kategori, lalu empat penanda:
   - `respons_dinilai` = sudah direspons, atau `now()` melewati batas respons;
   - `respons_tepat` = `responded_at` ≤ batas respons;
   - `selesai_dinilai` (hanya yang tidak ditolak) = sudah selesai, atau `now()` melewati batas selesai;
   - `selesai_tepat` = `resolved_at` ≤ batas selesai.
3. Per instansi:
   - R = jumlah `respons_tepat` ÷ jumlah `respons_dinilai`;
   - S = jumlah `selesai_tepat` ÷ jumlah `selesai_dinilai`;
   - P = laporan `selesai` ÷ laporan tidak ditolak;
   - K = ((jumlah bintang + 15) ÷ (jumlah pemberi rating + 5)) ÷ 5;
   - nilai = 100 × (0,3 R + 0,3 S + 0,2 P + 0,2 K), dibulatkan satu desimal.

4. `sampel` = jumlah `respons_dinilai`. Jika kurang dari `app_config.sampel_min_rapor` (10), `huruf` = `null`.
5. `tingkat_penolakan` = ditolak (`tidak_valid` + `konten_melanggar`) ÷ semua laporan (tanpa duplikat dan dibatalkan). `peringatan_penolakan` = tingkat > 25%, **atau** tingkat ⩾ 10% dan > 2 × median tingkat penolakan instansi lain.
6. `huruf_asli`: A ⩾ 85, B 70–84,9, C 55–69,9, D 40–54,9, E < 40. Jika peringatan menyala dan `huruf_asli` A atau B, maka `huruf` = C; selain itu `huruf` = `huruf_asli`.
7. `median_respons_jam` = `percentile_cont(0.5)` dari selisih `responded_at − created_at` dalam jam, hanya untuk laporan yang sudah direspons.
8. `per_kategori` memakai langkah 2–3 untuk R, S, dan P per kategori.

8.2 `priority_factors` (SQL, C)

Acuan waktu adalah `now()`. Karena data seed digeser saat reset (bagian 4.4), hasilnya tetap konsisten setelah setiap reset.

Faktor	Nilai mentah per isu wilayah
urgensi	Tinggi 1,0 · Sedang 0,6 · Rendah 0,3 (dari kategori)
volume	Laporan terverifikasi (`diverifikasi` / `diproses`) ÷ penduduk × 10.000
dukungan	Jumlah `least(upvote per laporan, 10)` atas laporan terverifikasi ÷ penduduk × 10.000
umur	Rata-rata `least(umur_jam ÷ sla_selesai_jam, 2)` atas semua laporan aktif
kerentanan	`persen_miskin` kecamatan (nilai level kota)
kepadatan	`penduduk ÷ luas_km2`

Hanya isu dengan minimal satu laporan aktif (`menunggu`, `diverifikasi`, `diproses`) yang dikembalikan. Mode `crm`: volume dari total `crm_agregat` tiga bulan terakhir, dukungan 0, umur `null`.

8.3 `lib/scoring` (TypeScript, C)

```
export const BOBOT_AWAL: Bobot;          // 0,30 / 0,20 / 0,05 / 0,20 / 0,15
/ 0,10
export const PRESET: Record<'keselamatan' | 'kerentanan' | 'suara_warga',
Bobot>;

export function normalisasiBobot(b: Bobot): Bobot;               // bagi
dengan jumlah; error jika semua 0
export function hitungSkor(isu: FaktorIsu[], bobot: Bobot): SkorIsu[];
export function selisihPeringkat(resmi: SkorIsu[], simulasi: SkorIsu[]):
Record<string, number>;
export function kalimatKonsekuensi(resmi: SkorIsu[], simulasi: SkorIsu[], n?:
number): string;
export function kenapaBelum(isuId: string, isu: FaktorIsu[], bobotResmi:
Bobot,
                            status: StatusLaporan): KenapaBelum;
```

Langkah `hitungSkor`:

1. Normalisasi bobot. Jika `umur` bernilai `null` (mode crm), bobot umur dijadikan 0 lalu dinormalisasi ulang.
2. Ubah ke skala 0–1: urgensi langsung; umur dibagi 2; volume, dukungan, kerentanan, kepadatan memakai min-max antar isu yang sedang dinilai. Jika maksimum = minimum, nilainya 0 untuk semua isu dan faktornya dicatat di `faktor_tidak_membedakan`.
3. `kontribusi[f] = 100 × bobot[f] × nilai01[f]`; `skor` = jumlah kontribusi (dibulatkan satu desimal hanya untuk tampilan).
4. Urutkan menurun berdasarkan skor; jika seri, berdasarkan `laporan_terverifikasi` lalu `kode_wilayah`. Isi `peringkat` mulai 1.
5. `penjelasan` = "Skor {skor}: terutama karena {faktor 1} ({poin 1} poin) dan {faktor 2} ({poin 2} poin)." dari dua kontribusi terbesar.

Langkah `kenapaBelum`: hitung skor resmi, lalu `pendorong` = faktor dengan kontribusi terbesar; `penahan` = faktor dengan selisih terbesar antara `100 × bobot` dan kontribusinya; `per_preset` = peringkat isu di tiap preset. Jika status `menunggu`, `catatan` menjelaskan bahwa laporan belum dihitung di volume.

Faktor	Label di kalimat
urgensi	urgensi kategori
volume	banyak laporan terverifikasi
dukungan	dukungan warga

Faktor	Label di kalimat
umur	laporan lama melewati batas waktu
kerentanan	kerentanan wilayah
kepadatan	kepadatan penduduk

Dengan sekitar 170 isu dan enam faktor, `hitungSkor` cukup ringan untuk dijalankan ulang setiap slider bergeser.

9. Desain halaman frontend

Setiap halaman dipetakan ke komponen dan operasi di bagian 6, supaya A tahu persis data apa yang dipanggil di mana.

Halaman	Rute	Komponen utama	Operasi
Beranda	`/`	Hero, StatistikRingkas, PreviewPeta, TombolDemo, BannerSimulasi	R1, Q1, A1
Peta Laporan	`/peta`	PetaLaporan (cluster), FilterKategoriStatus, KartuLaporan	Q1, Q4, F1
Detail & Lacak	`/laporan/[kode]`	GaleriSebelumSesudah, Timeline, IndikatorSLA, TombolUpvote, PanelKenapaBelum, FormRating, TombolBatalkan	Q2, R4, R5, R7, R9, L1
Buat Laporan	`/lapor`	BannerBukanKanalResmi, PetaPin, UnggahFoto, DaftarLaporanSerupa	Q4, S1, R2, R3
Masuk / Daftar	`/masuk`	FormMasuk, FormDaftar, InfoVerifikasiEmail	Supabase Auth
Rapor Instansi	`/rapor`	KartuRapor, BadgePeringatanPenolakan, TabelPerKategori	R8
Cara Menilai	`/rapor/cara-menilai`	Konten statis rumus rapor	—

Halaman	Rute	Komponen utama	Operasi
Prioritas	`/prioritas`	PetaChoropleth, DaftarPeringkat, BarKontribusi, PanelSimulator, RiwayatBobot, FormSimpanBobot (Perencana)	R9, Q5, Q6, F1, R10, L1
Sumber Data	`/sumber-data`	Daftar dataset, lisensi, skenario simulasi	—
Laporan Instansi	`/admin/laporan`	TabelLaporanAdmin	Q3
Detail dan Ubah Status	`/admin/laporan/[kode]`	FormUbahStatus (status sesuai transisi, alasan tolak, catatan, rujukan duplikat, unggah bukti)	Q2, S1, R6

9.1 Aturan tampilan

- **Proteksi rute:** middleware Next.js mengarahkan tamu dari `/lapor` dan `/admin/*` ke `/masuk`, dan non-admin dari `/admin/*` ke Beranda. Ini hanya kenyamanan; keamanan sebenarnya tetap di RLS dan RPC.
- **Leaflet hanya di browser:** komponen peta dimuat dengan `dynamic(() => import(...), { ssr: false })`, karena Leaflet membutuhkan `window`.
- **Warna choropleth tetap:** skor dibagi lima kelas tetap (0–20, 20–40, 40–60, 60–80, 80–100), bukan kuantil. Dengan begitu warna hanya berubah kalau skornya memang berubah saat slider digeser.
- **Setiap pemanggilan data** punya tampilan memuat (skeleton), tampilan kosong, dan pesan error dari `lib/errors.ts`.
- **Label sumber data** ("Data simulasi" atau "Data terbuka CRM DKI, periode …") tampil di dekat setiap angka di Beranda, Peta, Rapor, dan Prioritas.
- **Badge status** selalu berisi teks dan ikon, tidak hanya warna.
- **Mobile-first:** semua alur warga diuji di lebar 360 px; tabel admin boleh digulir ke samping di ponsel.

10. Pipeline data dan generator seed

Semua data disiapkan oleh lima skrip Python yang dijalankan berurutan di laptop C, lalu dimuat ke database. Hasilnya disimpan di repo, jadi siapa pun bisa membangun ulang database dari nol.

10.1 Urutan skrip

1. **`scripts/01_wilayah.py`** — ambil GeoJSON kecamatan dari `data/raw/`, saring provinsi DKI (kode 31) tanpa Kepulauan Seribu, sederhanakan bentuknya sampai file di bawah 1 MB tanpa merusak batas. Keluaran: `public/data/kecamatan.geojson` (properti `kode_wilayah`, `nama`, `kota`) dan `data/processed/regions.csv`.
2. **`scripts/02_bps.py`** — gabungkan penduduk dan luas per kecamatan serta persentase miskin per kota memakai kode wilayah. Skrip gagal jika ada kecamatan tanpa nilai.
3. **`scripts/03_cek_crm.py`** — profil kolom dataset CRM DKI dan tulis ringkasan untuk tim. Jika layak, agregasi ke `data/processed/crm_agregat.csv`.
4. **`scripts/04_akun.py`** — buat akun lewat Supabase Admin API: 4 admin instansi, 1 Perencana, warga demo, admin demo, dan 40 akun warga simulasi (untuk pelapor dan upvote seed).
5. **`scripts/05_seed.py`** — generator laporan (10.2), keluaran `supabase/seed/seed.sql` yang mengisi schema `seed` dan `public`.
6. **Muat ke database** lewat `psql` dengan `DATABASE_URL` dari Supabase, atau SQL editor Supabase.

10.2 Desain generator seed

- **Deterministik:** `numpy.random.default_rng(2026)`. Menjalankan ulang selalu menghasilkan data yang sama.
- **Waktu:** `seed_anchor` = waktu skrip dijalankan, disimpan di `app_config`. Laporan tersebar 180 hari sebelum anchor, dengan porsi lebih besar di 60 hari terakhir supaya cukup banyak laporan aktif untuk skor prioritas.
- **Jumlah dan kategori:** 900 laporan. Proporsi awal (asumsi, diganti proporsi CRM jika datasetnya layak): jalan rusak 35%, sampah 30%, saluran 20%, lampu 15%.
- **Lokasi:** kecamatan dipilih berbobot penduduk; titik diambil acak di dalam poligon (rejection sampling di kotak batasnya).
- **Siklus hidup laporan:** untuk setiap laporan, tentukan ditolak atau tidak (alasan: 70% tidak valid, 20% duplikat, 10% konten melanggar), waktu respons, dan waktu selesai sesuai profil instansi. Status di titik anchor mengikuti waktu-waktu itu, sehingga sekitar 25–35% laporan masih aktif.
- **Foto:** diambil dari kumpulan kecil foto per kategori milik tim sendiri atau berlisensi bebas; lisensinya dicatat di halaman Sumber Data.
- **Upvote:** sebaran Poisson per laporan, dengan beberapa laporan sampai 30 upvote untuk memperlihatkan batas 10 di skor.
- **Rating:** sekitar 60% laporan selesai diberi rating, rata-ratanya mengikuti profil instansi.

Instansi (simulasi)	Peluang ditolak	Respons tepat waktu	Selesai tepat waktu	Rating rata-rata	Peran di demo
Dinas Bina Jalan	0,05	0,90	0,80	4,2	Contoh dinas baik
Dinas Kebersihan	0,10	0,84	0,63	3,8	Mendekati contoh hitungan PRD (B)
Dinas Tata Air	0,06	0,50	0,35	2,8	Nilai D, sering terlambat
Dinas Penerangan Jalan	0,32	0,95	0,90	4,5	Angka setara A, tertahan C karena penolakan

Angka di tabel adalah parameter awal, bukan hasil. C menyetelnya sampai rapor menghasilkan keempat skenario di kolom terakhir.

10.3 Kasus khusus untuk video

- Satu kelompok berisi 3 laporan jalan rusak aktif dalam radius 100 m di lokasi demo, supaya "laporan serupa" muncul saat merekam.
- Satu laporan milik warga demo yang masih aktif di isu dengan peringkat menengah, untuk memperagakan "Kenapa belum?".
- Satu laporan berstatus `diproses` milik Dinas Bina Jalan, untuk diselesaikan admin demo di depan kamera.

Di akhir, skrip menghitung ulang rapor dengan Python dan mencetak ringkasan huruf per instansi, supaya skenario bisa dicek sebelum dimuat ke database.

11. Kinerja, penanganan error, logging

Target kinerja disesuaikan dengan kebutuhan demo dan uji juri di ponsel, bukan skala produksi.

Area	Target	Cara mencapainya
Peta dengan sekitar 1.000 titik	Lancar digeser di ponsel kelas menengah	`leaflet.markercluster`; Q1 hanya mengambil kolom yang dibutuhkan
Batas kecamatan	File di bawah 1 MB	Disederhanakan saat impor; disajikan sebagai file statis lewat CDN Vercel
Simulator bobot	Peringkat berubah di bawah 100 ms per geseran	`hitungSkor` di browser; data faktor diambil sekali per kunjungan

Area	Target	Cara mencapainya
Rapor Instansi	Di bawah 1 detik	Agregasi SQL dengan indeks; jika lambat, ganti ke materialized view yang diperbarui saat reset
Foto	Di bawah 300 KB per foto	Sisi terpanjang diperkecil ke 1600 px, simpan sebagai WebP kualitas 0,8
Query laporan	Tanpa full scan	Index GIST `lokasi`, index komposit status–kategori–wilayah, index timeline per laporan

11.1 Penanganan error

- Error dari RPC selalu berupa kode di bagian 6.2 dan ditampilkan sebagai pesan bahasa Indonesia, tidak pernah sebagai teks teknis mentah.
- Validasi form di browser memakai skema yang sama dengan aturan server (misalnya dengan `zod`), tapi server tetap jadi penentu akhir.
- Gagal jaringan atau unggah foto: isian form tidak hilang, dan ada tombol "Coba lagi".
- Halaman detail dengan kode yang tidak ada menampilkan halaman 404 bersahabat dengan kolom pencarian kode.

11.2 Logging

- API route: log Vercel. `reset-demo` mencatat jumlah baris yang dihapus dan disalin ulang.
- Database: log Supabase untuk error RPC dan penolakan RLS.
- Log tidak boleh memuat email, nama, atau isi laporan.

12. Deployment dan lingkungan

Satu project Supabase dan satu project Vercel dipakai bersama oleh tiga lingkungan. Ini cukup untuk lomba, dengan syarat perubahan skema hanya masuk lewat pull request yang di-merge B.

Lingkungan	Frontend	Database	Dipakai untuk
Lokal	`npm run dev`	Project Supabase yang sama	Ngoding harian

Lingkungan	Frontend	Database	Dipakai untuk
Preview	Vercel preview otomatis per PR	Project Supabase yang sama	Review PR dan uji di ponsel
Produksi	Vercel dari branch `main`	Project Supabase yang sama	Link untuk juri dan rekaman video

12.1 Setup awal (B, hari pertama)

1. Buat project Supabase di region Singapura (terdekat), aktifkan ekstensi PostGIS.
2. Auth: aktifkan **Confirm email**; isi Site URL dengan domain Vercel dan tambahkan `http://localhost:3000` ke daftar redirect.
3. Buat bucket `foto-laporan` dan `bukti` beserta kebijakannya (bagian 5.4).
4. Hubungkan repo ke Vercel, isi variabel lingkungan (bagian 3.2), tambahkan `vercel.json` berisi jadwal cron.
5. Jalankan migrasi `0001` – `0005`, lalu skrip data dan seed (bagian 10).

12.2 Migrasi

File SQL ditulis di `supabase/migrations/`, diterapkan dengan Supabase CLI (`supabase db push`) atau SQL editor, dan selalu dicatat di PR. File yang sudah diterapkan tidak diedit; perubahan berikutnya selalu file baru.

12.3 Jebakan paket gratis

- **Email bawaan Supabase sangat dibatasi** dan ditujukan untuk uji coba, sehingga email verifikasi bisa tidak terkirim ke juri. Sebelum link dibagikan, pasang SMTP sendiri (misalnya layanan email transaksional dengan paket gratis), dan tetap andalkan tombol akun demo.
- **Project gratis Supabase bisa dijeda** jika lama tidak ada aktivitas. Buka aplikasi atau dashboard secara rutin, terutama selama masa penilaian 12–20 November dan menjelang final.
- **Cron di paket gratis Vercel** hanya untuk jadwal harian dan waktunya tidak presisi; cukup untuk reset demo.

12.4 Cadangan untuk final onsite

Aplikasi tetap butuh internet untuk Supabase, jadi cadangan utamanya adalah rekaman video demo lengkap, kumpulan screenshot setiap halaman, dan hotspot pribadi. Menjalankan Supabase lokal lewat Docker bersifat opsional dan hanya jika ada waktu.

13. Rencana pengujian

Tiga lapis pengujian: tes otomatis untuk rumus, daftar cek manual untuk hak akses, dan uji alur lengkap mengikuti naskah video. Rumus diuji paling ketat karena paling sering ditanya juri.

13.1 Tes otomatis rumus (Vitest, C)

Dijalankan dengan `npm test` di setiap PR, idealnya otomatis lewat GitHub Actions. Tes rapor memakai versi TypeScript dari rumus yang sama dengan SQL, lalu hasil SQL dicocokkan dengan data seed.

Kasus	Masukan	Hasil yang diharapkan
Contoh hitungan PRD	R 0,84 · S 0,63 · P 0,72 · K 0,75	Nilai 73,5, huruf B
Peringatan penolakan	Tingkat penolakan 35%, angka setara A	`huruf_asli` A, `huruf` C
Dua dinas curang	Tingkat penolakan 5%, 6%, 30%, 30%	Kedua dinas 30% kena peringatan (batas tetap 25%)
Data belum cukup	Sampel 8	`huruf` = null
Belum ada rating	0 pemberi rating	K = 0,6
Bobot nol semua	Semua slider 0	Error, skor tidak dihitung
Faktor tidak membedakan	Kerentanan sama di semua isu	Nilai 0 untuk semua, faktor tercatat di `faktor_tidak_membedakan`
Konsistensi kontribusi	Isu mana pun	Jumlah kontribusi = skor (selisih ≤ 0,1)
Batas upvote	Satu laporan dengan 50 upvote	Dihitung 10
Preset	Ketiga preset	Total bobot 1,0
Mode crm	Umur bernilai null	Bobot umur 0, sisanya dinormalisasi ulang

13.2 Daftar cek hak akses (B, setelah 7 November)

- [ ] Tamu tidak bisa membaca tabel `reports` langsung, hanya lewat view
- [ ] Warga tidak bisa membatalkan laporan milik warga lain

- [ ] Admin Dinas Bina Jalan tidak bisa mengubah laporan Dinas Kebersihan
- [ ] Admin demo tidak bisa mengubah laporan dari warga sungguhan
- [ ] Warga dan admin instansi tidak bisa memanggil `save_weights`
- [ ] Upvote laporan sendiri ditolak; upvote kedua kali membatalkan dukungan
- [ ] Rating sebelum Selesai ditolak; rating kedua ditolak
- [ ] Isi laporan "Konten melanggar" tidak muncul di `v_laporan_publik`
- [ ] `SUPABASE_SERVICE_ROLE_KEY` tidak ada di kode yang terkirim ke browser
- [ ] Unggahan ke folder milik pengguna lain di Storage ditolak

13.3 Uji alur lengkap (semua, 8–9 November)

Ikuti naskah video di PRD bagian 13 dari awal sampai akhir, sekali di lebar 360 px dan sekali di desktop, setelah reset seed. Setiap langkah yang gagal atau terasa lambat dicatat sebagai bug. Uji ini diulang setiap kali sebelum merekam video.

14. Keterlacakan kebutuhan

Setiap kebutuhan fungsional di PRD bagian 6 punya tempat di desain ini. Jika ada FR yang berubah, baris di tabel ini menunjukkan bagian kode mana yang ikut terdampak.

FR	Ringkasan	Diwujudkan oleh
A1	Daftar, login, verifikasi email	Supabase Auth, `/masuk`, bagian 5.1
A2	Akun seed dan akun demo	`04_akun.py`, A1 `demo-login`
A3	Hak akses ditegakkan server	RLS (5.2), aturan RPC (5.3), middleware (9.1)
B1	Form, foto, EXIF, peringatan, banner	`/lapor`, S1, R3, bagian 5.4
B2	Kecamatan dan instansi otomatis	R3 (`ST_Contains`)
B3	Laporan serupa	R2, DaftarLaporanSerupa
B4	Kode unik dan status awal	R3
B5	Upvote	R4, primary key `upvotes`
B6	Batalkan oleh pelapor	R5
C1	Detail, foto sebelum/sesudah, SLA, "Kenapa belum?"	Q2, `v_laporan_publik`, PanelKenapaBelum, L1 `kenapaBelum`
C2	Cari dengan kode	Q2, kolom cari di header

FR	Ringkasan	Diwujudkan oleh
C3	Peta publik, Ditolak tetap tampil	Q1, PetaLaporan
C4	Rating	R7
D1	Daftar laporan instansi	Q3, `v_laporan_admin`
D2	Ubah status sesuai aturan	R6
D3	Timeline tercatat	`report_updates`, `v_timeline_publik`
D4	Hanya laporan instansi sendiri	R6 (`BUKAN_INSTANSI`), RLS `reports`
E1	Rapor publik dengan sampel, periode, per kategori	R8, KartuRapor, TabelPerKategori
E2	Definisi metrik	`/rapor/cara-menilai`
E3	Data belum cukup	R8 (`huruf` = null)
E4	Tingkat penolakan, peringatan, batas C	R8 (bagian 8.1 langkah 5–6)
F1	Skor prioritas	R9 + L1 `hitungSkor`
F2	Dashboard publik dan simulator	`/prioritas`, PanelSimulator, `selisihPeringkat`
F3	Kontribusi, nilai mentah, kalimat penjelasan	BarKontribusi, `SkorIsu.penjelasan`
F4	Bobot oleh Perencana, alasan, riwayat publik	R10, `weight_changes`, Q6
F5	Override *(sebaiknya ada)*	`priority_overrides`, `set_override`
G1	Seed simulasi yang bisa di-reset	`05_seed.py`, schema `seed`, R11, A2
G2	Impor wilayah dan BPS	`01_wilayah.py`, `02_bps.py`
G3	Halaman Sumber Data dan label sumber	`/sumber-data`, aturan label (9.1)

15. Keputusan desain dan pertanyaan terbuka

Keputusan di bawah diambil demi waktu 17 hari dan kemudahan menjelaskan ke juri. Masing-masing bisa ditinjau ulang setelah lomba.

Keputusan	Alternatif yang ditolak	Alasan
Supabase, tanpa server backend terpisah	Server sendiri (Express, Go)	Auth, storage, dan RLS siap pakai; menghemat beberapa hari kerja
Semua penulisan lewat RPC	Insert langsung + RLS	Aturan bisnis (transisi, duplikat, batas) terpusat dan bisa dites
Data publik lewat view	RLS saja	RLS tidak bisa menyembunyikan kolom; view menyembunyikan `user_id` dan konten melanggar
Skor dihitung di browser	Dihitung di SQL	Simulator harus instan; rumus mudah dites dan dijelaskan
Batas kecamatan sebagai file statis	Diambil dari database	Tidak pernah berubah; cepat lewat CDN
Riwayat bobot sebagai snapshot	Satu baris per faktor	Lebih mudah ditampilkan sebagai riwayat keputusan
Waktu seed digeser saat reset	Waktu acuan tetap di rumus	Angka demo konsisten tanpa logika waktu khusus di rumus
Satu project Supabase untuk semua lingkungan	Dev dan produksi terpisah	Cukup untuk lomba; risiko dikendalikan lewat aturan migrasi
Warna choropleth kelas tetap	Kelas kuantil	Perubahan warna mencerminkan perubahan skor yang nyata

Pertanyaan terbuka

- [ ] Dataset CRM DKI layak atau tidak, yang menentukan dipakai atau tidaknya mode `crm` (C, 27 Okt)
- [ ] Penyedia SMTP untuk email verifikasi (B, sebelum link dibagikan)
- [ ] Sumber foto seed dan lisensinya (C)
- [ ] Nama subdomain Vercel (B)
- [ ] Konfirmasi stack final di rapat 26 Oktober, jika ada anggota yang jauh lebih menguasai stack lain